Import Libraries & Load Dataset

```
# Importing required libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Loading the dataset

df = pd.read_csv('/content/traffic.csv')

# Display first 5 rows
df.head()
```

	event	date	country	city	artist	album	track	isrc	linkid
0	click	2021-08-21	Saudi Arabia	Jeddah	Tesher	Jalebi Baby	Jalebi Baby	QZNWQ2070741	2d896d31-97b6-4869-967b-1c5fb9cd4bb8
1	click	2021-08-21	Saudi Arabia	Jeddah	Tesher	Jalebi Baby	Jalebi Baby	QZNWQ2070741	2d896d31-97b6-4869-967b-1c5fb9cd4bb8
2	click	2021-08-21	India	Ludhiana	Reyanna Maria	So Pretty	So Pretty	USUM72100871	23199824-9cf5-4b98-942a-34965c3b0cc2
3	click	2021-08-21	France	Unknown	Simone & Simaria, Sebastian Yatra	No Llores Más	No Llores Más	BRUM72003904	35573248-4e49-47c7-af80-08a960fa74cd
		2021-				Jalebi	Jalebi		2d896d31-97b6-

```
# Check dataset information

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 226278 entries, 0 to 226277
Data columns (total 9 columns):
 #   Column   Non-Null Count   Dtype
---  ------   --------------   -----
 0   event    226278 non-null  object
 1   date     226278 non-null  object
 2   country  226267 non-null  object
 3   city     226267 non-null  object
 4   artist   226241 non-null  object
 5   album    226273 non-null  object
 6   track    226273 non-null  object
 7   isrc     219157 non-null  object
 8   linkid   226278 non-null  object
dtypes: object(9)
memory usage: 15.5+ MB
```

```
# Check number of rows and columns

df.shape
```

```
(226278, 9)
```

```
# Display column names

df.columns
```

```
Index(['event', 'date', 'country', 'city', 'artist', 'album', 'track', 'isrc',
       'linkid'],
      dtype='object')
```

```
# Check missing values

df.isnull().sum()
# Check duplicate records

df.duplicated().sum()
```

```
np.int64(103711)
```

Data Cleaning

```
# Convert date column into datetime format

df['date'] = pd.to_datetime(df['date'], errors='coerce')

# Check the datatype
df.dtypes
```

	0
event	object
date	datetime64[ns]
country	object
city	object
artist	object
album	object
track	object
isrc	object
linkid	object

dtype: object

```
# Check missing values percentage

missing_values = df.isnull().sum()

missing_values
```

	0
event	0
date	0
country	11
city	11
artist	37
album	5
track	5
isrc	7121
linkid	0

dtype: int64

```
# Fill missing categorical values

categorical_columns = [
    'country',
    'city',
    'artist',
    'album',
    'track',
    'isrc'
]

for col in categorical_columns:
    df[col] = df[col].fillna('Unknown')
```

```
# Remove duplicate records

df = df.drop_duplicates()

# Check dataset shape after cleaning

df.shape
```

```
(122567, 9)
```

Basic Traffic Metrics

```
# Total number of website traffic events

total_events = df.shape[0]

print("Total Traffic Events:", total_events)
```

```
Total Traffic Events: 122567
```

```
# Proxy session count

proxy_sessions = df.shape[0]

print("Proxy Sessions:", proxy_sessions)
```

```
Proxy Sessions: 122567
```

```
# Count unique link IDs as proxy users

proxy_users = df['linkid'].nunique()

print("Proxy Users:", proxy_users)
```

```
Proxy Users: 3839
```

```
# Count different event types

event_counts = df['event'].value_counts()

event_counts
```

event	count
pageview	73360
click	32499
preview	16708

dtype: int64

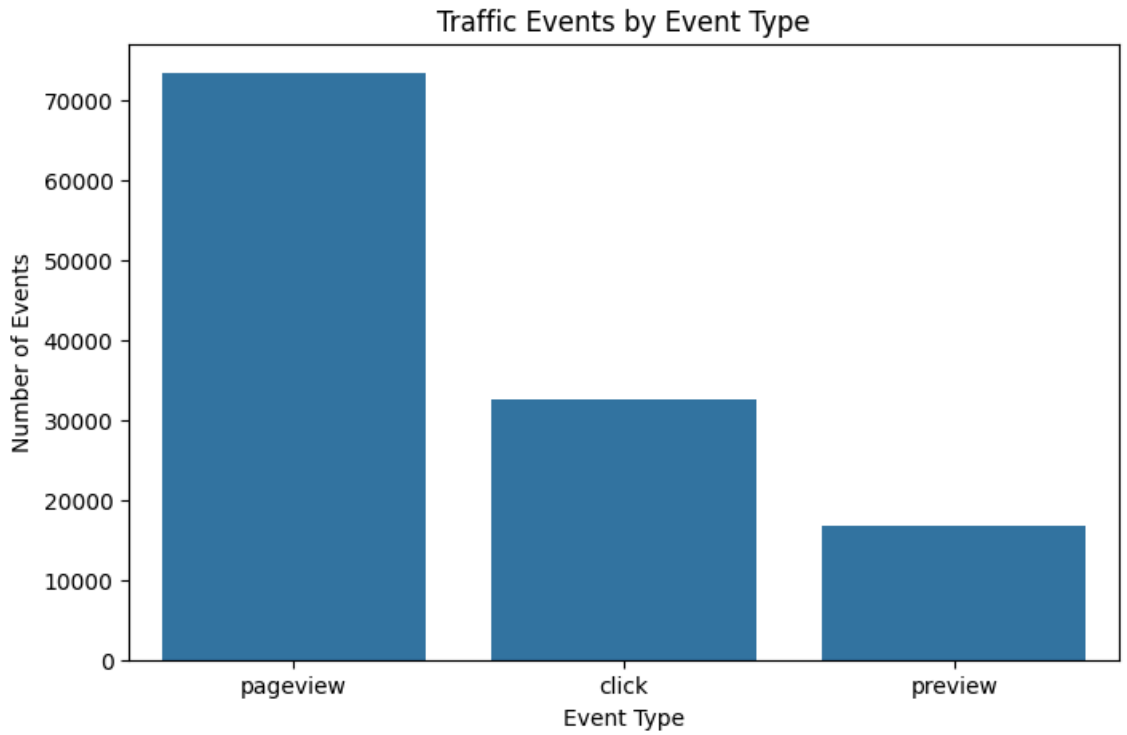
```
# Visualize event types

plt.figure(figsize=(8,5))

sns.countplot(
    data=df,
    x='event',
    order=df['event'].value_counts().index
)

plt.title('Traffic Events by Event Type')
plt.xlabel('Event Type')
plt.ylabel('Number of Events')

plt.show()
```



Bounce Rate Analysis

```
# Count events per link

events_per_link = df.groupby('linkid').size()

# Identify single-event links

single_event_links = events_per_link[events_per_link == 1]

# Calculate proxy bounce rate

proxy_bounce_rate = (
    len(single_event_links) / len(events_per_link)
) * 100

print("Proxy Bounce Rate:",
      round(proxy_bounce_rate, 2), "%")
```

```
Proxy Bounce Rate: 35.56 %
```

```
# Calculate first and last activity date for each link

activity_duration = df.groupby('linkid')['date'].agg(
    ['min', 'max']
)

# Calculate activity duration in days

activity_duration['duration_days'] = (
    activity_duration['max'] -
    activity_duration['min']
).dt.days

# Calculate average activity duration
```

```
average_duration = activity_duration['duration_days'].mean()

print(
    "Average Proxy Activity Duration:",
    round(average_duration, 2),
    "days"
)
```

```
Average Proxy Activity Duration: 0.83 days
```

Traffic Trend Analysis

```
# Count traffic events by date

daily_traffic = df.groupby('date').size()

daily_traffic.head()
```

	0
date	
2021-08-19	21156
2021-08-20	18536
2021-08-21	16701
2021-08-22	16927
2021-08-23	16415

dtype: int64

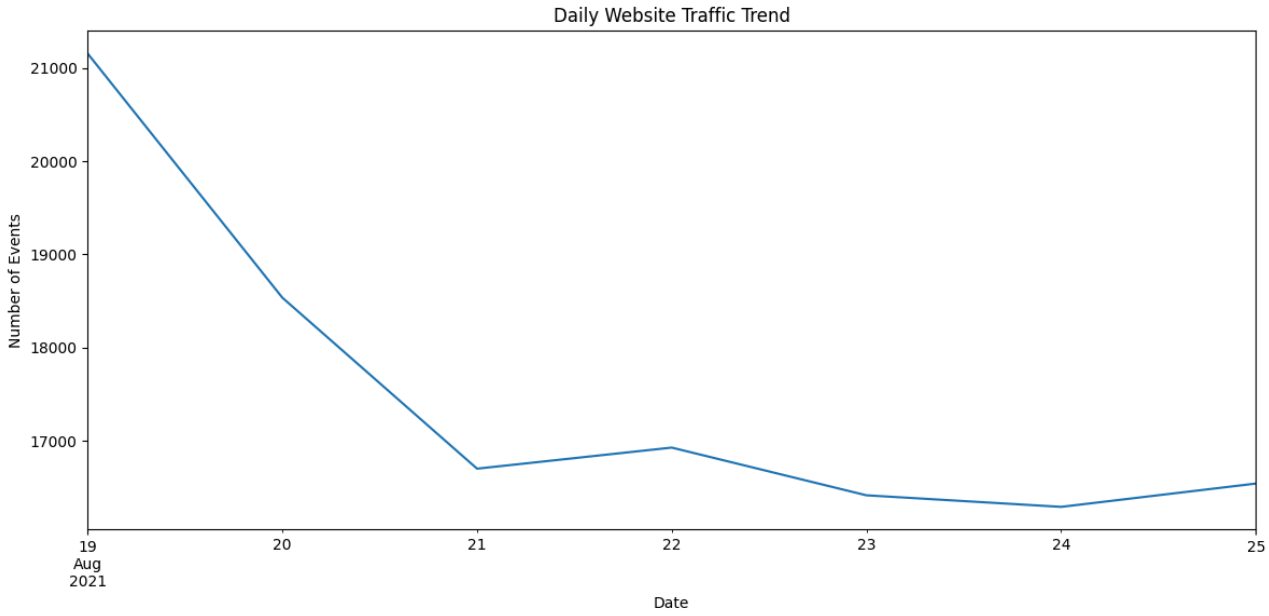
```
# Visualize daily traffic trend

plt.figure(figsize=(14,6))

daily_traffic.plot()

plt.title('Daily Website Traffic Trend')
plt.xlabel('Date')
plt.ylabel('Number of Events')

plt.show()
```




```
# Find top tracks/content

top_content = (
    df['track']
    .value_counts()
    .head(10)
)

top_content
```

track	count
Jalebi Baby	8288
Beautiful	4037
Beautiful Day	3951
Late At Night	3059
ily (i love you baby) (feat. Emilee)	2956
Calabria (feat. Lujavo & Nito-Onna)	2865
So Pretty	2830
Candy Shop	2397
Summer of Love (Shawn Mendes & Tainy)	2108
Build a Bitch	2072

dtype: int64

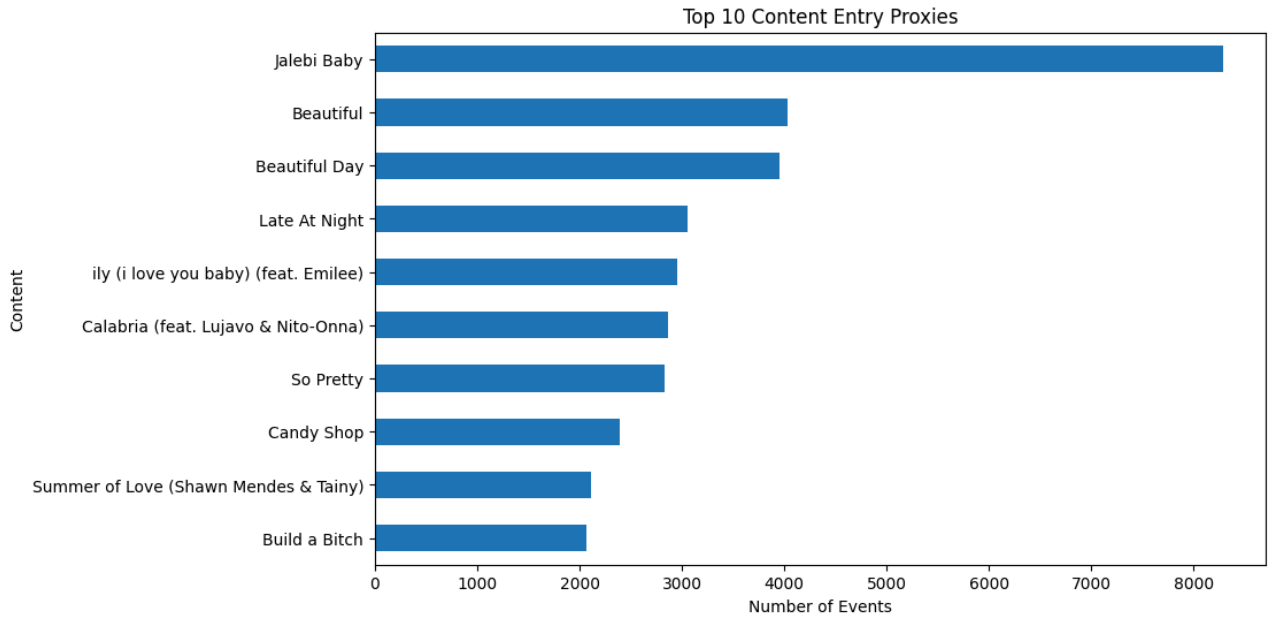
```
# Visualize top content

plt.figure(figsize=(10,6))

top_content.sort_values().plot(
    kind='barh'
)
```

```
plt.title('Top 10 Content Entry Proxies')
plt.xlabel('Number of Events')
plt.ylabel('Content')

plt.show()
```




```
# Count content interaction frequency

exit_content = (
    df.groupby('track')
      .size()
      .sort_values(ascending=False)
      .head(10)
)

exit_content
```

	0
track	
Jalebi Baby	8288
Beautiful	4037
Beautiful Day	3951
Late At Night	3059
ily (i love you baby) (feat. Emilee)	2956
Calabria (feat. Lujavo & Nito-Onna)	2865
So Pretty	2830
Candy Shop	2397
Summer of Love (Shawn Mendes & Tainy)	2108
Build a Bitch	2072

dtype: int64

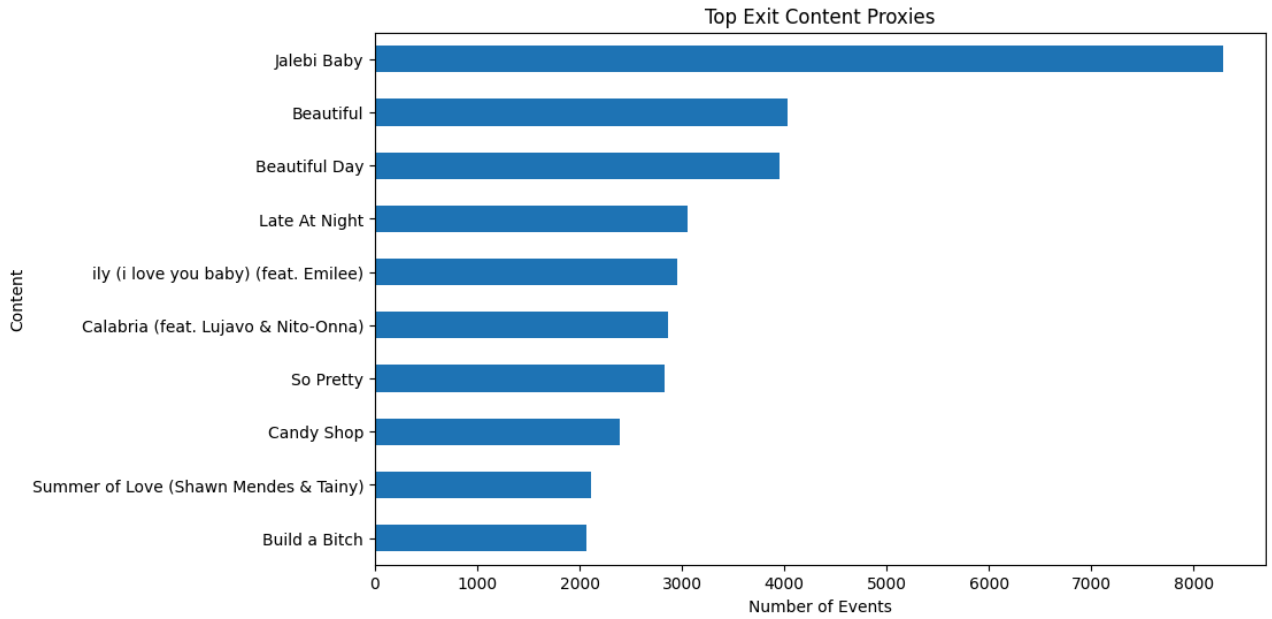
```
# Visualize top exit content proxies

plt.figure(figsize=(10,6))

exit_content.sort_values().plot(
    kind='barh'
)

plt.title('Top Exit Content Proxies')
plt.xlabel('Number of Events')
plt.ylabel('Content')

plt.show()
```




```
# Find top countries generating traffic

top_countries = (
    df['country']
    .value_counts()
    .head(10)
)

top_countries
```

country	count
United States	28664
India	18689
France	10565
Saudi Arabia	7682
United Kingdom	5095
Germany	4017
Canada	2784
Pakistan	2633
Iraq	2444
Turkey	2399

dtype: int64

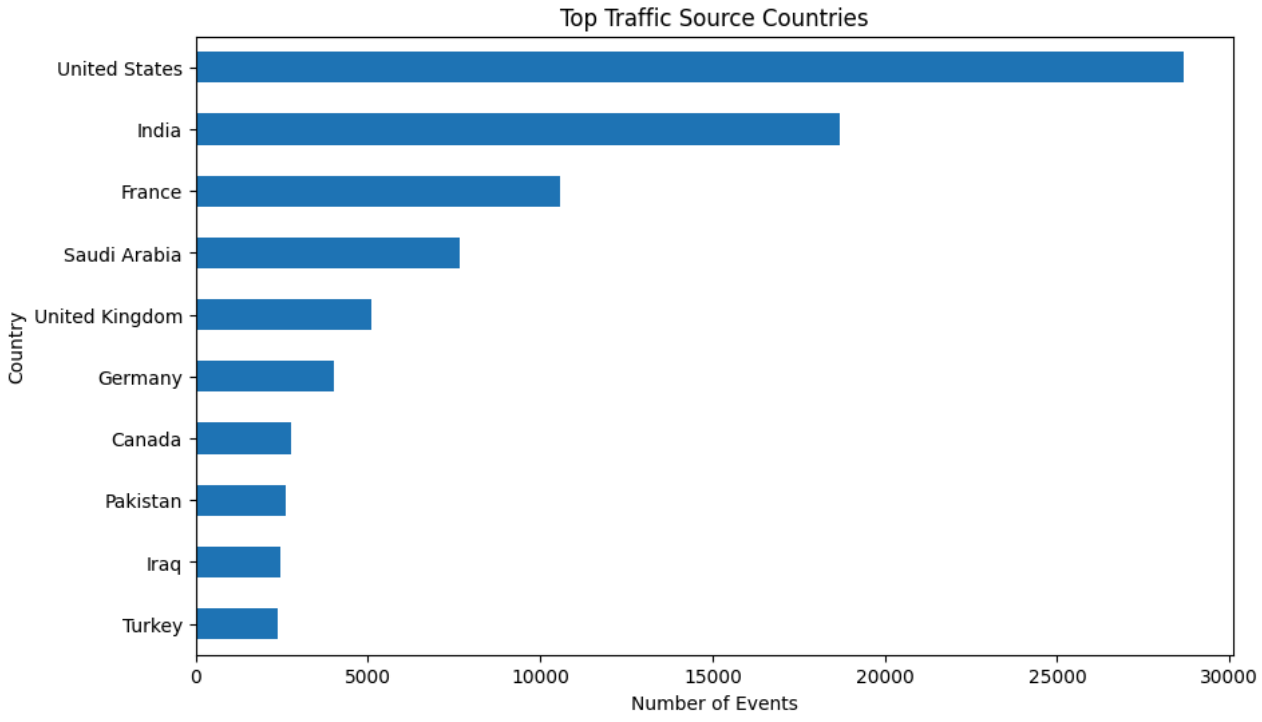
```
# Visualize top countries

plt.figure(figsize=(10,6))

top_countries.sort_values().plot(
    kind='barh'
)

plt.title('Top Traffic Source Countries')
plt.xlabel('Number of Events')
plt.ylabel('Country')

plt.show()
```



```
# Find top cities by traffic

top_cities = (
    df['city']
    .value_counts()
    .head(10)
)

top_cities
```

city	count
Unknown	8802
Jeddah	2497
Riyadh	2232
Hyderabad	1088
Dammam	1002
Delhi	884
Jaipur	849
Lucknow	837
Kuwait City	816
Ahmedabad	808

dtype: int64

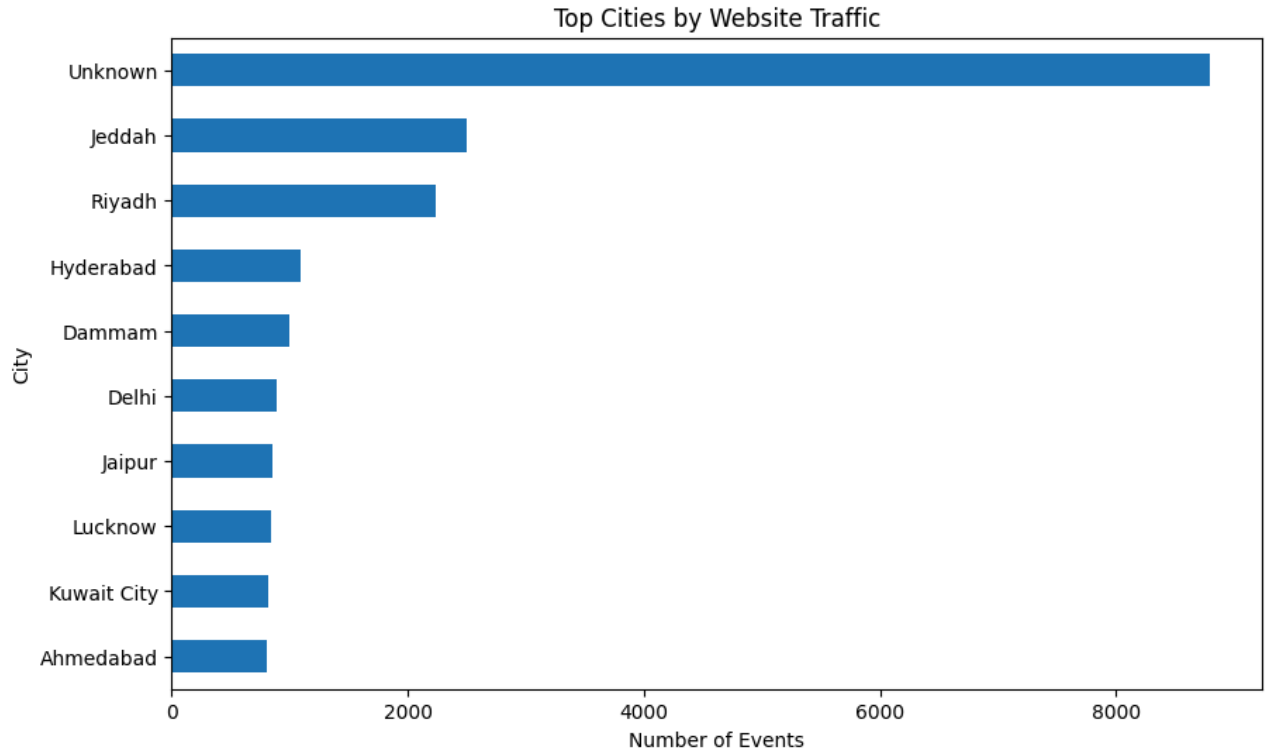
```
# Visualize top cities

plt.figure(figsize=(10,6))

top_cities.sort_values().plot(
    kind='barh'
)

plt.title('Top Cities by Website Traffic')
plt.xlabel('Number of Events')
plt.ylabel('City')

plt.show()
```




```
# Create event transition table

df_sorted = df.sort_values(
    by=['linkid', 'date']
)

df_sorted['next_event'] = (
    df_sorted
    .groupby('linkid')['event']
    .shift(-1)
)

event_flow = pd.crosstab(
    df_sorted['event'],
    df_sorted['next_event']
)

event_flow
```

event \ next_event	click	pageview	preview
click	27966	2495	2036
pageview	2147	67163	213
preview	299	2009	14400

Next steps: Generate code with `event_flow` | New interactive sheet

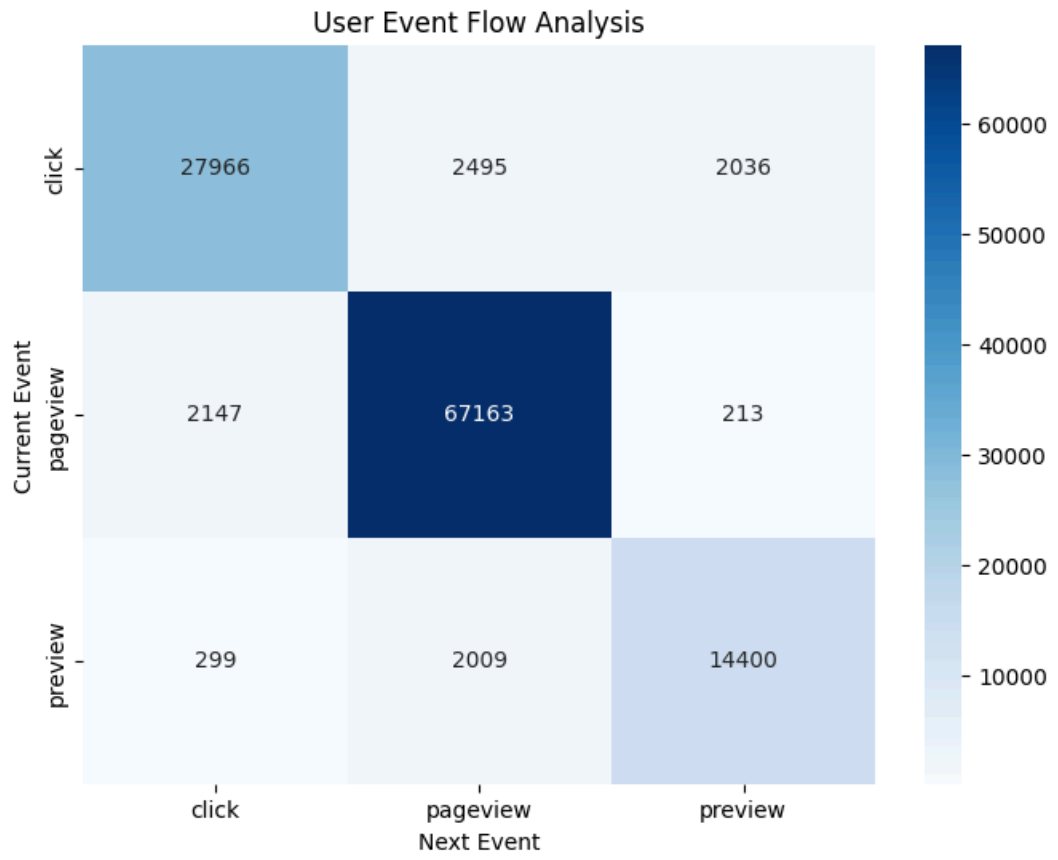
```
# Visualize event flow

plt.figure(figsize=(8,6))
```

```
sns.heatmap(
    event_flow,
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.title('User Event Flow Analysis')
plt.xlabel('Next Event')
plt.ylabel('Current Event')

plt.show()
```




```
# Create KPI summary

kpi_summary = pd.DataFrame({
    'Metric': [
        'Total Events',
        'Proxy Sessions',
        'Proxy Users',
        'Proxy Bounce Rate (%)',
        'Average Proxy Activity Duration (Days)'
    ],
    'Value': [
        total_events,
        proxy_sessions,
        proxy_users,
        round(proxy_bounce_rate, 2),
        round(average_duration, 2)
    ]
})

kpi_summary
```